DMDD 6210 Database Design Document – Project Part 1

1. Team Introduction

Name	Responsibility
Darshan Aswathappa	Develops ER diagrams and logical models and Perform Validation
Nireeksha Huns	Implements database schema and query optimization

Team comprises professionals with diverse experience in data modeling, application development, and testing. Together, we aim to design a robust and scalable database solution tailored to the project's requirements.

2. Project Name

Project Title: Hospital Management System (HMS)

3. Problem Statement

Hospitals and healthcare facilities manage a high volume of patient interactions daily spanning appointments, admissions, prescriptions, billing, and insurance claims. In the absence of a structured database system, these operations are often handled through disconnected spreadsheets, paper-based records, and siloed software tools. This leads to critical failures across the care delivery lifecycle.

- Patient records are duplicated or incomplete, causing misdiagnosis risks and delayed care.
- Appointment scheduling lacks coordination with doctor availability, resulting in overbooking, missed slots, and untracked rescheduling history.
- Admission and bed management has no real-time visibility into bed occupancy, allowing the same bed to be assigned to multiple patients simultaneously.
- Billing is disconnected from clinical events, invoices are generated manually after discharge, leading to missed charges for outpatient visits and insurance miscalculations.
- Prescription records exist outside the context of the visit or admission they belong to, creating gaps in medication history and audit trails.
- Insurance validation is performed manually, with no automated enforcement of coverage dates, plan activity, or applicable coverage caps.

Approach

Requirement Gathering: Identify all data entities, relationships, cardinality constraints, and reporting needs across five core modules, Patient Management, Doctor Management, Appointment Management, Admission & Bed Management, and Billing & Payments. Capture business rules explicitly.

Conceptual Modeling: Construct a high-level Entity-Relationship (ER) diagram representing 15 core entities and their associations. Validate cardinality at every relationship boundary, one-to-one, one-to-many, and optional associations, to accurately reflect real-world operational constraints.

Logical Modeling: Convert the conceptual model into a relational schema. Assign primary keys to every entity, define foreign key references across related tables, and introduce junction attributes where required. Apply nullable vs. NOT NULL designations based on business rules.

Normalization: Apply normalization rules (1NF–3NF) to eliminate redundancy and improve data integrity.

- 1NF - Eliminate repeating groups; ensure all columns are atomic
- 2NF - Remove partial dependencies; schedule details live in DoctorSchedule, not duplicated inside Appointment
- 3NF - Remove transitive dependencies; insurance coverage amounts are stored on Insurance, not re-derived on Billing

Implementation: Develop the schema in a target RDBMS (PostgreSQL recommended for native support of partial indexes, GiST exclusion constraints, and deferred foreign keys). Define all tables, constraints, indexes, and triggers in migration scripts. Seed reference data for departments, room types, and insurance plans.

Testing & Optimization: Validate referential integrity across all FK relationships. Test each enforced business rule with positive and negative cases. Review query execution plans on high-frequency paths (appointment lookup by doctor/date, billing by patient, bed availability). Add composite indexes on frequently filtered column combinations

Expected Outcome

Minimal redundancy: 3NF normalization eliminates duplicate data across all 15 entities.

Referential integrity: Foreign keys, partial unique indexes, and deferred constraints enforce consistency at the database level.

Audit traceability: AppointmentHistory and Payment tables provide full change and transaction logs.

Real-time bed visibility: Bed.status is synchronized with Admission.status via triggers, ensuring the schema is always consistent

Ease of querying and reporting: Normalized schema with well-defined relationships supports efficient joins for patient history, revenue reporting, and occupancy analytics.

5. Database Conceptual Model (ER Diagram Description)

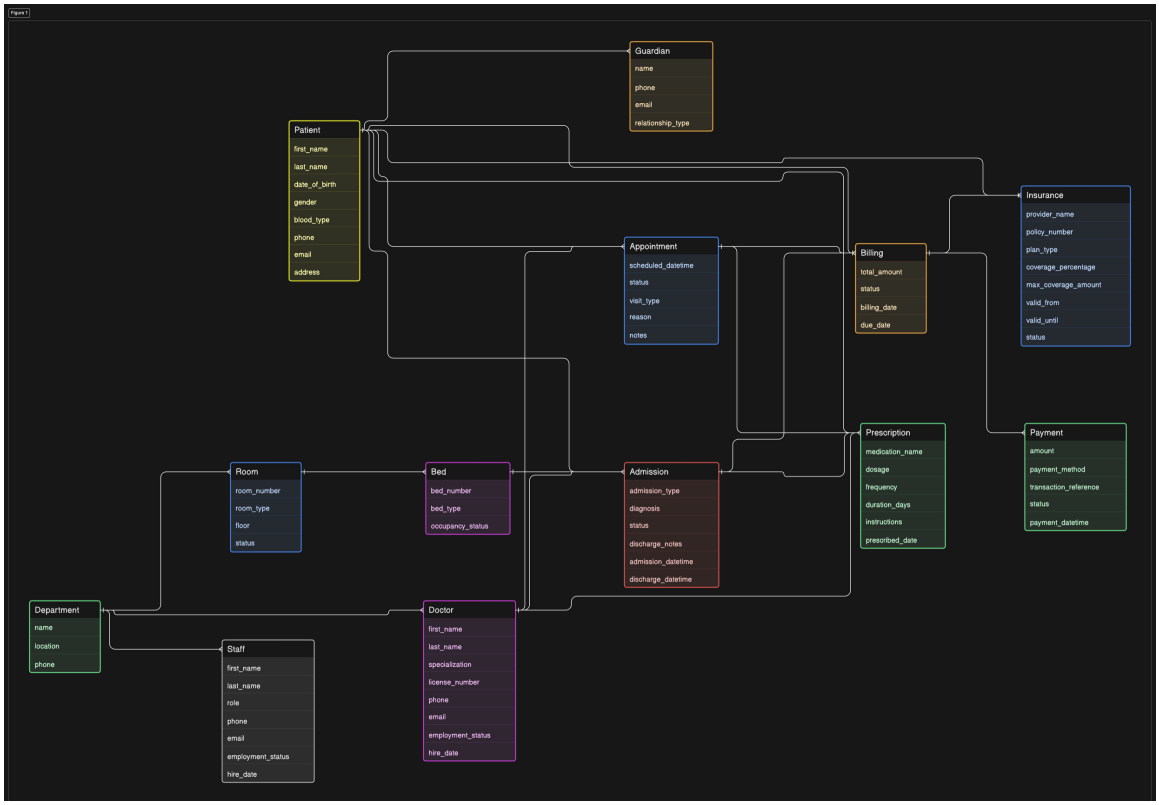

Figure 1: Conceptual ER Diagram. [url]

Patient represents an individual receiving medical care at the hospital. It captures personal and contact details such as name, date of birth, gender, blood type, phone, email, and address. A patient is the central entity in the system, they book appointments, undergo admissions, receive prescriptions, hold insurance policies, have guardians, and incur billing charges.

Guardian represents a person legally or personally responsible for a patient, such as a parent or next of kin. It stores the guardian's name, contact information, and the type of relationship they have with the patient. Each patient can have one or more guardians, but each guardian record belongs to exactly one patient.

Insurance represents a health insurance policy held by a patient. It includes the provider name, policy number, plan type, coverage percentage, maximum coverage amount, validity period, and current status. A patient can hold multiple insurance policies, and each policy is linked to a single patient. A billing record can claim against one insurance policy.

Department represents an organizational unit within the hospital such as Cardiology, Orthopedics, or Emergency. It stores the department name, physical location, and contact number. Each department employs multiple doctors and staff members, contains multiple rooms, and is headed by exactly one doctor.

Doctor represents a licensed medical professional employed by the hospital. It captures personal details, specialization, license number, contact information, employment status, and hire date. A doctor belongs to one department, and one doctor can serve as the head of that department. Doctors conduct appointments, handle admissions, and write prescriptions.

Staff represents non-physician hospital employees such as nurses, technicians, and administrative personnel. It stores name, role, contact details, employment status, and hire date. Each staff member belongs to exactly one department, and a department can employ many staff members.

Room represents a physical space within the hospital used for patient care. It includes the room number, type (e.g., ICU, general ward, operation theatre), floor, and current availability status. Each room belongs to one department and contains one or more beds.

Bed represents an individual bed within a room. It tracks the bed number, bed type, and current occupancy status. Each bed belongs to exactly one room, and a bed can accommodate multiple admissions over time (one at a time, sequentially).

Appointment represents a scheduled visit between a patient and a doctor. It records the scheduled date and time, status, visit type (e.g., in-person, teleconsultation), reason for the visit, and clinical notes. Each appointment is booked by one patient with one doctor. An appointment can result in one or more prescriptions and generates exactly one billing record.

Admission represents a patient being formally admitted to the hospital for inpatient care. It captures the admission type (e.g., emergency, planned), diagnosis, status, discharge notes, and the admission and discharge timestamps. Each admission involves one patient, is handled by one doctor, and is accommodated in one bed. An admission can result in one or more prescriptions and generates exactly one billing record.

Prescription represents a medication order issued by a doctor for a patient. It includes the medication name, dosage, frequency, duration, special instructions, and the date it was prescribed. A prescription is linked to one patient and one doctor, and can originate from either an appointment or an admission.

Billing represents a financial charge generated for services rendered. It stores the total amount, payment status, billing date, and due date. Each billing record is tied to one patient, may be linked to an appointment or an admission, and can claim against one insurance policy. A single billing record can be settled through one or more payments.

Payment represents an individual financial transaction made against a billing record. It captures the amount paid, payment method, transaction reference, status, and timestamp. Each payment belongs to exactly one billing record, and a billing record can have multiple payments (e.g., partial payments or split payments).

Relationships:

A Patient *has* one or more Guardians.

A Patient *holds* one or more Insurance policies.

A Patient *books* one or more Appointments.

A Patient *undergoes* one or more Admissions.

A Patient *receives* one or more Prescriptions.

A Patient *incurs* one or more Billing records.

A Department *employs* one or more Doctors.

A Department *employs* one or more Staff members.

A Department *contains* one or more Rooms.

A Department *is headed by* exactly one Doctor.

A Doctor *conducts* one or more Appointments.

A Doctor *handles* one or more Admissions.

A Doctor *writes* one or more Prescriptions.

An Appointment *results in* one or more Prescriptions.

An Appointment *generates* exactly one Billing record.

An Admission *results in* one or more Prescriptions.

An Admission *generates* exactly one Billing record.

A Room *contains* one or more Beds.

A Bed *accommodates* one or more Admissions.

A Billing record *claims* against exactly one Insurance policy.

A Billing record *is settled by* one or more Payments.

6. Relational Model (Logical Design)

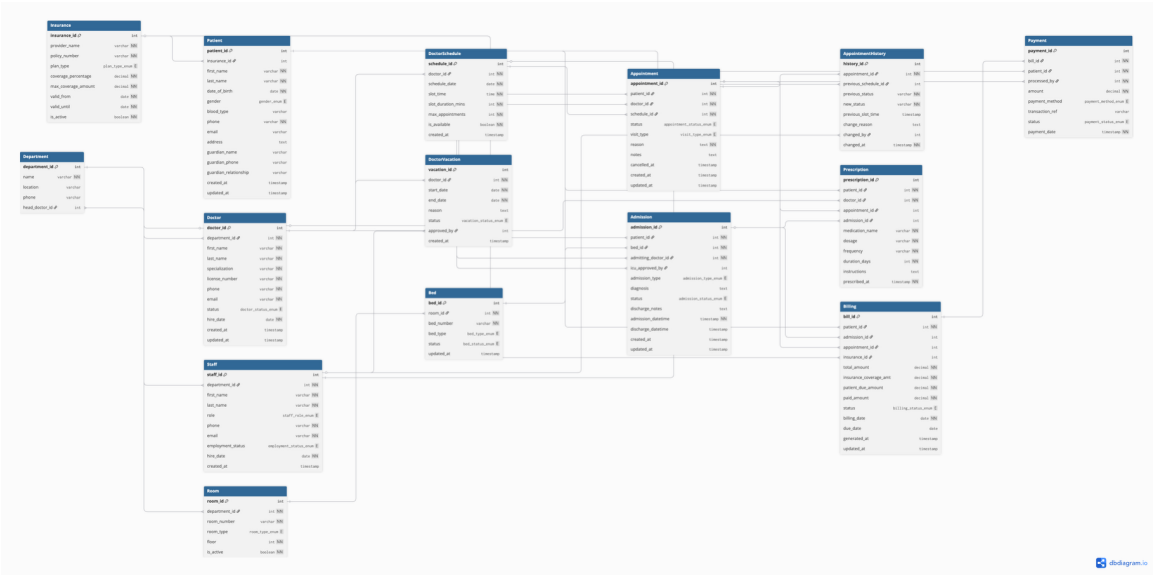

Figure 2: Logical Design [URL]

Table Name	Primary Key	Foreign Keys	Attributes
Patient	patient_id	insurance_id	first_name, last_name, date_of_birth, gender, blood_type, phone, email, address, guardian_name, guardian_phone, guardian_relationship, created_at, updated_at
Insurance	insurance_id	—	provider_name, policy_number, plan_type, coverage_percentage, max_coverage_amount, valid_from, valid_until, is_active
Department	department_id	head_doctor_id	name, location, phone
Doctor	doctor_id	department_id	first_name, last_name, specialization, license_number, phone, email, status, hire_date, created_at, updated_at
Staff	staff_id	department_id	first_name, last_name, role, phone, email, employment_status, hire_date, created_at
DoctorSchedule	schedule_id	doctor_id	schedule_date, slot_time, slot_duration_mins, max_appointments, is_available, created_at
DoctorVacation	vacation_id	doctor_id, approved_by	start_date, end_date, reason, status, created_at
Appointment	appointment_id	patient_id, schedule_id	status, visit_type, reason, notes, cancelled_at, created_at, updated_at
AppointmentHistory	history_id	appointment_id, previous_schedule_id, changed_by	previous_status, new_status, previous_slot_time, change_reason, changed_at

Table Name	Primary Key	Foreign Keys	Attributes
Room	room_id	department_id	room_number, room_type, floor, is_active
Bed	bed_id	room_id	bed_number, bed_type, is_maintenance, updated_at
Admission	admission_id	patient_id, bed_id, admitting_doctor_id, icu_approved_by	admission_type, diagnosis, status, discharge_notes, admission_datetime, discharge_datetime, created_at, updated_at
Prescription	prescription_id	appointment_id, admission_id	medication_name, dosage, frequency, duration_days, instructions, prescribed_at
Billing	bill_id	admission_id, appointment_id	total_amount, insurance_coverage_amt, status, billing_date, due_date, generated_at, updated_at
Payment	payment_id	bill_id, processed_by	amount, payment_method, transaction_ref, status, payment_date

Note: All derived/computed columns (patient_id in Billing/Payment/Prescription, doctor_id in Appointment/Prescription, insurance_id in Billing, paid_amount in Billing, bed status) have been removed per strict 3NF. These values are recovered via recommended views (v_appointment, v_prescription, v_billing, v_payment, v_bed_status).

7. Normalization Steps

First Normal Form (1NF)

Rule: Every column must contain atomic (indivisible) values. No repeating groups, no multi-valued fields, and every row must be uniquely identifiable.

Guardian information for minor patients could have been stored as a single composite field (e.g., guardian_details = "John Doe | 9876543210 | Father"). Instead, it was split into three atomic columns, guardian_name, guardian_phone, guardian_relationship, each holding a single value. Similarly, patient contact information (phone, email, address) is stored in separate columns rather than a single contact_info blob. No table contains repeating groups: for instance, Prescription stores one medication per row rather than packing multiple medications into a single record, and Billing captures one billing event per row rather than bundling line items into a comma-separated field. Each table uses a surrogate integer primary key (patient_id, doctor_id, etc.) to guarantee row uniqueness.

Result: All 15 tables have unique rows with atomic, single-valued attributes. No field requires string parsing to extract meaningful data, and no row contains embedded lists or repeating groups.

Second Normal Form (2NF)

Rule: Remove partial dependencies, every non-key attribute must depend on the entire primary key, not just a part of it.

Action: All 15 tables use single-column surrogate primary keys (patient_id, doctor_id, schedule_id, etc.), which structurally eliminates partial dependencies, there is no composite key for a non-key attribute to depend on only part of. Design decisions were made to keep it this way: rather than modelling Prescription with a composite key of (appointment_id, medication_name), it was given its own prescription_id, and rather than keying Bed as (room_id, bed_number), it uses a standalone bed_id. In an early draft, Appointment carried a redundant appointment_datetime column alongside schedule_id. Since the datetime is fully determined by the schedule slot (DoctorSchedule.schedule_date + slot_time), it was removed to prevent divergence between Appointment and its source of truth in DoctorSchedule.

Result: All non-key attributes in every table depend on the full primary key of that table. No partial dependencies exist because no composite primary keys exist.

Third Normal Form (3NF)

Rule: Remove transitive dependencies, no non-key attribute should depend on another non-key attribute. Every non-key attribute must depend on the key, the whole key, and nothing but the key.

Action: Insurance coverage details (coverage_percentage, max_coverage_amount, valid_from, valid_until, is_active) describe the insurance plan itself, not the patient. If stored on the Patient table, these attributes would be transitively dependent: Patient.coverage_percentage -> depends on -> Patient.insurance_id -> depends on -> Insurance.insurance_id. This transitive chain was eliminated by housing all plan details exclusively in the Insurance table. Likewise, room_type and floor describe the room, not the bed, so they remain in Room, and Bed only holds a room_id FK. department_id on Doctor points to Department; department attributes like location and phone are never duplicated onto Doctor.

Beyond these structural separations, several stored derived values were also removed to achieve strict 3NF. patient_id was eliminated from Billing, Payment, and Prescription, it is derivable through FK chains (e.g., Billing -> Admission -> Patient). doctor_id was removed from Appointment and Prescription for the same reason (derivable via schedule_id -> DoctorSchedule.doctor_id). insurance_id was removed from Billing because it is reachable through the patient's record (Billing -> Admission/Appointment -> Patient.insurance_id). paid_amount was removed from Billing because it is computable as the sum of completed Payment rows for that bill. Bed.status was replaced with a non-derivable is_maintenance flag, since bed occupancy is determinable from the existence of an active Admission row, storing it separately would have created a transitive dependency on Admission.status. All removed convenience columns are recovered through recommended database views (v_appointment, v_prescription, v_billing, v_payment, v_bed_status).

One documented exception exists: AppointmentHistory.previous_slot_time is technically derivable from previous_schedule_id, but is intentionally stored as an audit snapshot. Schedule rows may be mutated after the fact, and this column preserves the original value at the time of change, a standard, defensible practice for immutable audit tables.

Result: Each non-key attribute in every table depends only on the primary key of that table. No indirect, chained, or computed dependencies exist anywhere in the schema, with the single documented exception of the audit snapshot in AppointmentHistory.

8. Topic (Module Chosen)

Appointment Management Module

The Appointment Management module focuses on handling the complete lifecycle of patient appointments within the hospital system. This includes enabling patients to schedule, reschedule, and cancel appointments with doctors based on availability and department specialization.

In this module, we will be responsible for designing and implementing the core logic that connects patients, doctors, and appointment records. This involves ensuring that each appointment is properly associated with a patient and a doctor, while maintaining accurate scheduling details such as date, time, visit type, and reason for consultation.

Additionally, we will work on managing appointment statuses (e.g., scheduled, completed, cancelled) and ensuring data consistency when appointments lead to further processes such as prescriptions or billing. The module will also support handling real-world constraints like avoiding scheduling conflicts and maintaining proper tracking of appointment-related information.

Overall, the goal of this module is to provide a reliable and efficient system for managing patient visits, improving coordination between patients and healthcare providers, and supporting downstream processes within the hospital system.

9. Conclusion

This Hospital Management System database design ensures structured, reliable storage of clinical and administrative data across all five operational modules, patient management, doctor management, appointment scheduling, admission and bed management, and billing and payments. By applying normalization through Third Normal Form (3NF), the schema eliminates redundancy, resolves transitive dependencies, and guarantees that every attribute is anchored to its correct entity. The clear separation of concerns across 15 well-defined tables, enforced through primary keys, foreign key constraints, partial unique indexes, exclusion constraints, and business-rule triggers, ensures referential integrity at every layer of the system. The dual-context billing model, synchronized bed availability, and audit-ready appointment history provide a foundation for accurate real-time reporting and regulatory compliance. Together, these design decisions produce a scalable, maintainable, and extensible schema capable of supporting future enhancement, such as telemedicine integration, multi-facility expansion, and advanced analytics for patient outcomes and resource utilization.